

EmberDHT

Protocol Specification

WIRE VERSION

4 (min 4)

SOFTWARE

Ember 1.6.3

TRANSPORT

Noise_IK / Noise_XX

DATE

September 2026

EmberDHT is Ember's native Kademlia overlay: a signed, encrypted, server-less distributed hash table that Ember nodes run among themselves, in parallel with the eMule KAD network and eD2K servers. This document specifies the identity model, Noise transport, wire frames, routing table, record formats, lookup and publish algorithms, bootstrap paths, and abuse limits as implemented in Ember 1.6.3. It is written from the code, not from comments.

Contents

FOUNDATIONS

1	Introduction	4
2	Architecture	6
3	Identity and cryptography	7
4	Transport	9

OVERLAY

5	Kademlia parameters and routing	12
6	Wire protocol	14
7	Records: keywords and sources	21
8	Iterative lookup and search	25
9	Publish and replication	27

OPERATION

10	Bootstrap and joining	29
11	NAT, observed address, and firewalled peers	30
12	Abuse resistance	31
13	Maintenance	33
14	Persistence	34

REFERENCE

15	Comparison with eMule KAD	35
16	Current limits and planned work	36
A	Constant tables	38
B	Code map	42

This specification describes the protocol as implemented in Ember 1.6.3 (`EMBER_DHT_VERSION = 4` , `EMBER_DHT_MIN_VERSION = 4`). Two breaking changes have landed since wire version 2: v3 reshaped contact lists and `FOUND_VALUE` , and v4 bound a frame signature to the session it arrives on. Both are marked in place. Everything added

since without a bump — query constraints, record media, the callback and endorsement pair, channel records, and the advertised version range — is marked **ADDITIVE**, meaning a peer that does not speak it parses the frame correctly and ignores the addition. Remaining work, including the dormant native transfer path, is called out where it affects interoperability. Design notes that are not yet wire-stable live in `docs/ember-dht.md`.

SECTION 1

Introduction

What EmberDHT is, what it is not, and the constraints that shaped it.

Ember is a modern eMule-compatible P2P client. Beside speaking KAD and eD2K, every Ember node also runs **EmberDHT**: its own encrypted Kademlia overlay, used to find other Ember nodes, publish shared files, and resolve download sources without a directory server, tracker, or shipped seed list.

1.1 Purpose

EmberDHT answers three questions among Ember-capable peers:

1. **Who is on the overlay?** — routing-table contacts bound to Ed25519 / X25519 keypairs.
2. **Which files exist under a name?** — keyword records, findable by search.
3. **Who can serve this file?** — source records keyed on the eD2K file hash.

Content still moves over the eMule client-to-client wire. EmberDHT discovers the source; eD2K transfers the bytes. A 256 KiB chunk protocol with a BLAKE3 hash tree exists in `ember/transfer.rs` but is not yet wired in.

1.2 Relationship to KAD and eD2K

EmberDHT is a *second network*, not a replacement. It is always on (`ember_native_enabled`). The settings switch remains visible but cannot be turned off. The overlay rides the same UDP socket as KAD (port 4672 by default) and is demultiplexed by a two-byte magic, so one forwarded port serves both stacks.

KAD and eD2K are also how a cold Ember node *finds* the overlay. There is no Ember bootstrap server. Join paths are the KAD rendezvous key, peers noticed in ordinary KAD traffic, Ember-capable eD2K sessions, gossip, and a persisted contact file. The Friends rendezvous server has no role in DHT bootstrap: a server-hosted pool would hand the operator an identity-to-IP roster of every participant.

1.3 Design principles

- **Cryptographic identity.** A node ID is not chosen; it is `BLAKE3(Ed25519 public key)[..16]`. Gossip that claims an ID without the matching key is discarded.
- **Signed everything.** Every DHT frame is Ed25519-signed over its header and payload. Every stored record is signed by its publisher. A `DhtMessage` only exists in verified form.
- **Encrypted transport.** After the magic bytes, the datagram is Noise (ChaCha20-Poly1305, BLAKE2s). Cleartext DHT never leaves the host.
- **No central roster.** Seed lists, DNS SRV, and rendezvous-hosted bootstrap pools are explicitly not planned.
- **Fit the datagram.** Replies are packed to an unfragmented 1400-byte budget. An oversized reply that the peer drops before decryption looks like success to the sender and silence to the receiver.

- **Tighten, never loosen.** Abuse limits start permissive on a near-empty table and converge on KAD-strict values as verified contacts grow. Tightening refuses new admissions; it never evicts what an earlier tier allowed.

1.4 Document conventions

Integers on the DHT frame header and most payload lengths are **little-endian**. Socket ports in *contact-list* and *PONG* addresses are **big-endian**; TCP/UDP ports inside a signed source record are little-endian. Byte strings are concatenated in the order shown. Hexadecimal constants use a `0x` prefix. Time constants are wall-clock seconds unless noted as `Instant` (process-monotonic).

NORMATIVE SOURCE

Where this document and a comment disagree, the constants and encoders in `src-tauri/src/network/ember/` win. Protocol constants live in `dht/mod.rs` and `dht/messages.rs`; publish/search drivers and maintenance live in `network/mod.rs`.

2. Architecture

Ember frames share the KAD UDP socket. The network task demultiplexes on magic, decrypts through the Noise session table, then offers the plaintext to the control decoder first and the DHT decoder second. The two namespaces cannot alias: control version `0xC1` sits far outside the DHT version range.

APPLICATION	Search UI · Library publish badges · Ember Network page · status-bar gauges
DRIVERS	Publish / search / maintenance in <code>network/mod.rs</code> — keyword & source cycles, KAD bridge, rendezvous lookup, streaming results
DHT ENGINE	IO-free protocol core — routing table, store, iterative search, signed frame build/parse. Transport-agnostic: it consumes decrypted bytes and returns frames to encrypt.
RECORDS	Keyword (<code>0x01</code>) and source (<code>0x02</code>) blobs, Ed25519-signed by the publisher, stored under 16-byte BLAKE3 keys
TRANSPORT	Noise_IK when the peer's static X25519 is known · Noise_XX for first contact with a stateless retry cookie · ChaCha20-Poly1305 · BLAKE2s
UDP	Shared KAD socket · magic <code>0xEB 0x3E</code> · default port 4672 · max datagram 4096 B, replies packed to 1400 B

Figure 1. EmberDHT stack. The engine is deliberately free of sockets so the protocol is unit-testable without a live network.

2.1 What the overlay does and does not do

DOES	DOES NOT
Find Ember nodes and keep a verified routing table	Move file bytes (that is still eD2K c2c)
Publish and look up keyword and source records	Replace KAD or eD2K; it runs beside them
Carry BLAKE3 integrity digests on records	Host a bootstrap roster or seed list
Let firewalled nodes publish via buddy <code>PROXY_STORE</code> , consume via <code>CALLBACK_REQ</code> , and start the Ember broker for <code>LowID↔LowID</code>	Guarantee a working relay (that still needs admitted ERAT candidates)
Refuse incompatible wire versions at the version byte	Negotiate an upgrade or tell the user why a peer was dropped

2.2 Always on

Profiles that still had the overlay off are turned on at load. Publishing starts once the node has contacts. Search, the Ember Network page, and the status bar wait for a **verified** contact before showing Connected — gossip in the table is not a join. A fresh node therefore reports **Connecting** for a minute or two while the first maintenance cycle runs the KAD bridge, rather than a fault. After the join timeout with still-zero verified peers, Search shows the muted no-peers hint.

3. Identity and cryptography

3.1 Keypairs

Every Ember node holds two static keypairs, generated once and persisted:

KEY	ALGORITHM	ROLE
Ed25519	ed25519-dalek, strict verification	Node identity, frame signatures, record signatures, node ID derivation
X25519	static Noise key	Transport session (IK when known, XX when not)

3.2 Node ID

```
node_id[16] = BLAKE3(ed25519_public_key[32])[0..16]
```

The 128-bit ID is compatible with Ember's existing `ember_hash` field and is cryptographically bound to the signing key. Since wire version 3 a contact list carries no `node_id` at all: the receiver derives it from the Ed25519 bytes beside it and refuses the contact if the key is not a valid curve point. Before v3 the ID was transmitted and then re-derived and compared, so the only thing those 16 bytes could do was disagree with the key they travelled with. Deriving rather than checking is the primary defence against routing-table poisoning: a peer cannot name a contact under an ID it does not control, because it does not get to name the ID.

PROOF OF POSSESSION

The BLAKE3 binding is an *offline consistency check*, not proof the peer holds the private key. Anyone who observed a legitimate public key on the wire can replay the pair. Frame and record signatures are the proof-of-possession for DHT work. Friend-level privileges use a separate challenge-response over a fresh nonce and are out of scope here.

3.3 Frame signature

`encode_message` serializes version, type, request id, sender id, public key, payload length, and payload, then appends a 64-byte Ed25519 signature over every preceding byte. `decode_message` verifies that signature and the `sender_id = BLAKE3(pubkey)[..16]` binding *before* constructing a `DhtMessage`. An invalid frame never becomes a message object. This build always includes the public key on the wire.

WIRE VERSION 4 — SESSION BINDING

The signature also covers the sender's own Noise static key, which is *not* transmitted: the receiver already learned it from the handshake, so the frame is the same size on the wire and only the signature changes.

Without that binding a signed frame was a bearer token. It proved only that its `sender_id` had once signed those bytes, not that whoever delivered them was that sender — so anyone who had ever received a frame from Alice could replay it verbatim inside their own Noise session, and the receiver, which records a verified contact from the session's address and static key on every frame that decodes, would file Alice at the replayer's address with the replayer's key. The `noise_pub` pin then held that entry against the real Alice, and replaying kept it fresh so it never aged out.

A v3 signature cannot verify here and ours cannot verify there, so the version byte had to move with it. This is the change that makes a refusal at the version check preferable to a stream of "signature verification failed".

3.4 Record signature

A stored record is a publisher-signed blob. The signature covers the record body (type, hashes, size, publisher key, timestamp, name, and for sources the contact block). Storers re-send the identical bytes; they do not re-sign. Expiry is computed from the *signed* creation timestamp, so replication cannot extend a record's life.

4. Transport

Ember UDP payloads are told apart from KAD/eD2K by magic `0xEB 0x3E` . Everything after the magic is encrypted. Patterns:

- `Noise_IK_25519_ChaChaPoly_BLAKE2s` — peer’s static X25519 is already known (routing-table contact, previous session).
- `Noise_XX_25519_ChaChaPoly_BLAKE2s` — first contact, including the eD2K-bridge path where no static key is known.

4.1 Packet types

TYPE	NAME	ROLE
0x01	PKT_IK_INIT	IK handshake, initiator → responder
0x02	PKT_IK_RESP	IK handshake, responder → initiator
0x03	PKT_XX_MSG1	XX handshake message 1
0x04	PKT_XX_MSG2	XX handshake message 2
0x05	PKT_XX_MSG3	XX handshake message 3
0x06	PKT_XX_COOKIE	Stateless retry cookie (return-routability)
0x10	PKT_TRANSPORT	AEAD payload (DHT or control)

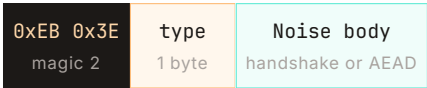


Figure 2. Outer UDP layout. Transport packets add an 8-byte nonce and 16-byte Poly1305 tag around the inner frame (27 bytes of transport overhead including magic and type).

4.2 Return-routability cookie

An unauthenticated XX msg1 can be forged off-path. The responder sends a **stateless retry cookie** (`PKT_XX_COOKIE`) once the unvalidated-handshake budget is spent, so a peer that does not know this packet type still completes first contact whenever the node is not under a flood. The cookie is only useful to a source address that can receive a UDP reply.

Inbound XX handshakes may briefly queue outbound traffic for that peer (honest simultaneous-open). After a 3-second grace the node dials the identity itself; once an IK initiator is pending, further inbound msg1s from that address are refused so a forged handshake cannot be renewed.

4.3 Session lifecycle

LIMIT	VALUE	WHY
SESSION_TIMEOUT	900 s	Must outlive the 600 s liveness-ping interval; 300 s forced a fresh handshake on every ping
MAX_SESSIONS	4096	LRU eviction when full
MAX_SESSIONS_PER_ADDR	4	Claimants coexist; keyed on (address, static key)
Pending handshakes	512	Caps unauthenticated work
MAX_EMBER_DATAGRAM_BYTES	4096	Hard drop before decryption; sender sees a "successful" send

Sessions are keyed on (address, static key), so claimants at one address coexist instead of ranking for a single live slot. Four matches the old 1-live-plus-3-shadow budget. A genuine first contact at an address already full of spoof sessions is kept; a named outgoing identity no longer discards another key at that address.

4.4 Inner namespace: control vs DHT

Control frames and DHT frames share one decrypted byte stream. The payload is offered to the control decoder first. Control version is 0xC1 so it can never collide with EMBER_DHT_VERSION (which counts up from 1). A historical bug: control version 1 made CONTROL_KIND_EXCHANGE_DATA (4) alias MSG_FOUND_NODE (4), and every iterative lookup stalled after its first hop.

KIND	VALUE	BODY
CONTROL_VERSION	0xC1	Leading byte of every control frame
PING / PONG	1 / 2	Transport keepalive (not DHT PING)
EXCHANGE_REQUEST	3	Empty; ask for UDP EPX payload
EXCHANGE_DATA	4	EPX wire payload, packed to the datagram budget

EPX IS NOT EMBERDHT

Ember Peer Exchange (opcode 0xF0 on the eMule extended protocol) is a separate mechanism: source lists exchanged with peers you are already transferring with. It works with the overlay switched off. This specification covers the DHT overlay only.

4.5 Size budgets

A receiver drops anything over 4096 bytes before decryption. Replies whose size we choose are packed to MAX_UNFRAGMENTED_DATAGRAM = 1400, matching the spirit of KAD's UDP_KAD_MAXFRAGMENT (1420). Fragmented UDP is dropped by a fair number of consumer NATs; a shorter answer that arrives beats a complete one that does not.

$$\text{MAX_UNFRAGMENTED_PAYLOAD} = 1400 - 27 - 120 = 1253 \text{ bytes}$$

27 = transport overhead (magic + type + nonce + tag). 120 = frame overhead (22-byte header + 32-byte public key + 2-byte length + 64-byte signature). `FOUND_VALUE` record blobs then have $1253 - 16 - 2 = 1235$ bytes after the key and count — about five typical keyword records per reply.

5. Kademlia parameters and routing

EmberDHT is a 128-bit XOR-metric Kademlia with a replacement cache, verified contacts, and network-size-adaptive diversity caps.

NODE ID 128 bit BLAKE3 of Ed25519 pubkey	K (BUCKET SIZE) 20 KAD uses 10	A (CONCURRENCY) 5 KAD FindNode uses 1	BUCKETS 128 One per ID bit
CONTACT TIMEOUT 600 s Then liveness ping	FAILED QUERIES 3 Then evict		

5.1 Distance and buckets

$$\text{distance}(a, b) = a \text{ XOR } b \quad \cdot \quad \text{bucket} = \text{index of highest set bit of distance (0...127)}$$

Bucket 127 holds contacts in the opposite half of the keyspace; bucket 0 holds the nearest neighbours. Node IDs are uniform, so occupancy is geometric: about half of all contacts fall in the last bucket and a quarter in the one before. Diversity caps are designed around that, not around a flat table.

5.2 Contact

A routing-table contact is:

FIELD	SIZE	NOTES
node_id	16	Derived from ed25519_pub; not carried on the wire since v3
addr	IPv4 or IPv6 + port	Port 0 is not admitted
noise_pub	32	X25519 static key for IK
ed25519_pub	32	Signing / identity key
last_seen	i64 unix	> 0 means verified
failed_queries	u8	Consecutive unanswered queries

Verified means we have heard a signed frame from the contact directly. Gossip from `FOUND_NODE` / `PEER_LIST` / `ANNOUNCE_PEER` and entries loaded from disk arrive with `last_seen = 0`. They are kept as leads but are not preferred for seeding lookups, answering peers, persisting, or computing network scale.

5.3 Replacement cache

Each bucket holds up to k contacts plus a k -entry replacement cache. When the bucket is full, a new contact is cached and the caller is asked to ping the oldest live entry. If that ping fails, `evict_and_replace` swaps in the newest cache entry. KAD in this codebase has no replacement cache.

5.4 Admission

A contact is refused if any of the following hold:

- Port is 0, or the address is filtered by the user's `ipfilter.dat`.
- Private / LAN / CGNAT addresses while "block private IPs" is on (same preference as KAD; toggling it on evicts already-admitted matches).
- The per-IP or per-subnet diversity cap for the current `NetworkScale` is full (see §12).
- The Ed25519 key is not a valid curve point, or `BLAKE3(key)[..16]` disagrees with the advertised ID.

5.5 Store proximity

A node accepts a STORE only if it is among the k closest contacts it knows of to the key — or if it knows fewer than k contacts, in which case it is among the k closest by definition. Distance is XOR against the local ID compared to the k -th closest contact. Gossip does not count: letting unverified leads into the comparison would let an attacker push the node out of its own neighbourhood.

SMALL-NETWORK TRAP

A previous check used "closer than our furthest contact in any bucket," which on a small network made every node trivially among the k closest to everything — until the table filled, at which point replication silently halved. The k -closest rule is what the code uses now.

6. Wire protocol

6.1 Versioning

CONSTANT	VALUE	MEANING
EMBER_DHT_VERSION	4	Wire version this build speaks
EMBER_DHT_MIN_VERSION	4	Oldest version this build can parse

Version 0 is invalid. A frame outside `[MIN, VERSION]` is refused at the version byte rather than becoming a malformed-payload counter that reads like packet loss.

VERSION	SHIPPED	WHAT CHANGED SHAPE
2	1.5.3	Batched store, its ack, contact-list trimming and payload limits all changed while the byte still read 1 — two peers announced the same version and then misparsed each other.
3	1.5.7	<code>node_id</code> dropped from wire contacts (87 → 71 bytes); <code>FIND_VALUE</code> gained <code>start_position</code> and <code>FOUND_VALUE</code> gained <code>next_position</code> and <code>total_available</code> . A v2 peer reads both at fixed offsets, so neither could be additive.
4	1.6.0	The sender's Noise static key joined the <i>signed</i> bytes without being transmitted (§3.3). No layout change; a v3 signature simply cannot verify here.

A change that only *adds* to the format can lower `MIN_VERSION` instead of raising both. Neither of the last two could: v3 moved fields a v2 peer reads at fixed offsets, and v4 changed what the signature covers.

WHY A BUMP PARTITIONS, AND WHAT NOW MITIGATES IT

The version byte is range-checked on *receive*, so raising `EMBER_DHT_VERSION` partitions the overlay regardless of where `MIN_VERSION` sits — the peer that refuses the frame is the one running the old range, and it cannot be reasoned with after the fact.

Since 1.6.3, `PING` and `PONG` carry the range their sender can decode (§6.5), so a future encoder can ask per peer rather than assume, and send the older shape to anyone who has not answered. That does not help against peers predating the advertisement itself, which is why the diagnostic counting how many contacts have advertised one is the number to watch before any further bump.

A refused frame is counted and split by direction, so a node can tell peers that are behind from peers that are ahead, and the Ember page raises a banner while older peers are being turned away. The peer that needs to update is by definition the one that cannot decode anything we send.

6.2 Frame layout

SIGNED DHT FRAME (AFTER NOISE DECRYPTION)

```

version u8 this build: 4
msg_type u8 see table below
request_id u32 LE correlation; 0 is reserved
sender_id 16 B BLAKE3(sender Ed25519)[..16]
sender_pub 32 B Ed25519 public key; always present in this build
payload_len u16 LE ≤ MAX_DELIVERABLE_PAYLOAD
payload N B message-specific
signature 64 B Ed25519 over every preceding byte plus the sender's own Noise static key (v4, not sent)

```

The encoder always includes the sender's 32-byte Ed25519 public key (`include_pub_key = true`). The decoder always expects it (`has_pub_key = true`) and refuses a frame that omits it, so a peer who has never seen us can verify the signature and learn our identity even inside an established Noise session. The size budget in §4.5 therefore always counts those 32 bytes. The encode/decode flags still exist in the API; production never takes the other branch.

Header with public key is 54 bytes before `payload_len`: version + type + request id + sender id + public key. Request id 0 is skipped by the allocator so it can never collide with "unset."

6.3 Message catalog

ID	NAME	DIRECTION	PAYLOAD
0x01	PING	→	optional advertised version range
0x02	PONG	←	optional observed SocketAddr, then optional advertised version range
0x03	FIND_NODE	→	target node ID (16)
0x04	FOUND_NODE	←	contact list
0x05	STORE_RECORD	→	key + record + record signature
0x06	STORE_ACK	←	key (16)
0x07	FIND_VALUE	→	1..8 keys + start_position, then an optional constraint block
0x08	FOUND_VALUE	←	key + next_position + total_available + records (or FOUND_NODE on miss)
0x09	ANNOUNCE_PEER	→	contact list (gossip dump)
0x0A	PEER_LIST	←	contact list
0x0B	PROXY_STORE	→	same body as STORE_RECORD
0x0C	PROXY_STORE_ACK	←	key (16) — buddy accepted, not a DHT STORE_ACK
0x0D	STORE_BATCH	→	up to 64 records for one destination
0x0E	STORE_BATCH_ACK	←	u64 bitmap of accepted positions
0x0F	CALLBACK_REQ	→	publisher id (16) + file hash (16) + searcher TCP port (2 LE) + crypt options (1) + searcher user hash (16) + callback token (16). Searcher → HighID buddy. The searcher's IP is the UDP source, not in the payload.
0x10	CALLBACK	→	file hash (16) + searcher IPv4 (4) + TCP port (2 LE) + crypt options (1) + searcher user hash (16) + callback token (16). Buddy → firewalled publisher: connect-and-serve this searcher.
0x11	CHANNEL_MSG	→	AEAD channel gossip body for overlay rooms. ADDITIVE
0x12	CHANNEL_RELAY	→	opaque CHANNEL_MSG envelope for a HighID hop to forward to a LowID target. ADDITIVE
0x13	BUDDY_ENDORSE_REQ	→	empty — the requester is the authenticated frame <code>sender_id</code> , and that is what the endorsement binds to. ADDITIVE
0x14	BUDDY_ENDORSE	←	buddy IPv4 (4) + UDP port (2) + noise_pub (32) + expiry (8) + signature (64). ADDITIVE

Every type from `0x0F` up is additive: a peer that does not speak one decodes it as `Unknown` and ignores it, because the header layout is unchanged. None of them cost a version bump.

NUMBERING IS GLOBAL, NOT PER BRANCH

The endorsement pair sits *after* the channel types rather than filling the gap before them. `BUDDY_ENDORSE_REQ` carries no payload and its decoder ignores the body, so sharing a number with `CHANNEL_MSG` would have had a channel frame decode cleanly on that arm and draw a signed endorsement in reply. Two features developed in parallel must not both claim the next free byte.

6.4 Contact list encoding

Used by `FOUND_NODE`, `ANNOUNCE_PEER`, and `PEER_LIST`. Contacts are already ordered closest-first; the encoder trims the tail to stay inside the unfragmented payload budget.

CONTACT LIST — WIRE VERSION 3 ONWARD

```
count u8 ≤ 20, then for each contact:
addr 1 + 4 or 16 + 2 BE  0x04 = IPv4, 0x06 = IPv6
noise_pub 32 B
ed25519 32 B
```

An IPv4 contact is $7 + 32 + 32 = 71$ bytes. At 1253 bytes of payload, **17 IPv4 contacts** fit, so a `FOUND_NODE` still never carries a full k-bucket of 20 — `MAX_CONTACTS_PER_RESPONSE` is 20 but is not reachable on IPv4.

Version 2 also transmitted a 16-byte `node_id` per contact, at 87 bytes each and 14 per reply. Every decoder re-derived the ID from the Ed25519 key beside it and compared, so the only thing those bytes could do was disagree with that key; dropping them bought most of a hop on a sparse table. This is the *wire* format only — `nodes_ember.dat` still stores an advisory ID per contact (§14.1), because changing the file format would cost every user their bootstrap set for no bandwidth saving.

The asker is excluded from a `FOUND_NODE` reply: they were just added to our table, their distance to themselves is zero, and leaving them in spent a slot of a size-capped response on the one contact they already have.

6.5 PING / PONG

`PONG` may carry the sender's observed address (the address we received the ping from), encoded the same way as a contact address: `0x04 / 0x06` + IP + port (big-endian). An empty `PONG` payload decodes as "no observation" for backward compatibility. Observed-IP voting is specified in §11.

Both frames may then carry a tag/len/value block advertising the wire versions their sender can decode.

ADDITIVE**ADVERTISED VERSION RANGE (TRAILING BLOCK)**

```
tag u8 0x01 = version range
len u8 2
min u8 oldest version the sender can parse
max u8 newest version the sender can parse
```

This is what lets a future encoder pick a frame shape per peer instead of assuming (§6.1). It is additive because no existing decoder looks here: `PING` discards its payload entirely, and `PONG` reads its address through a decoder that checks only a minimum length and reports what it consumed. The version byte

therefore must *not* move for this — the peers it exists to reach are exactly the ones that would refuse the frame carrying it.

Three properties are load-bearing. The block only ever trails a field a reader parses first, so a `PONG` with no observed address carries none: with nothing in front of it, an older build would read the tag byte as an address type and reject the whole frame. Absent, truncated and nonsensical (`min = 0` , or `min > max`) all read as “this peer said nothing” rather than as a range or an error, because an advisory field must not be able to drop a frame. And an unknown tag is skipped rather than fatal, which is what lets a later capability share the same block.

The claim rides inside the signed bytes, so it is exactly as forgeable as the rest of the frame and no more: a relay cannot rewrite what a peer says it speaks. A receiver holds the range per peer and forgets it when the peer leaves the routing table — it is worth what the session that proved it is worth, and one ping relearns it.

6.6 FIND_NODE / FOUND_NODE

FIND_NODE PAYLOAD

```
target 16 B node ID we want neighbours of
```

The responder returns the closest contacts it has, excluding the asker, packed as a contact list. Iterative use is specified in §8.

6.7 STORE_RECORD / STORE_ACK / PROXY_STORE

STORE_RECORD AND PROXY_STORE PAYLOAD

```
key 16 B
record_len u16 LE ≤ MAX_STORE_RECORD_BYTES
record N B publisher-signed body
record_signature 64 B Ed25519 over record only
```

`MAX_STORE_RECORD_BYTES` is the largest body that still fits in a `FOUND_VALUE` as the only blob (pack budget minus the 2-byte length prefix and 64-byte publisher signature). A larger body would store but never be served.

`STORE_ACK` and `PROXY_STORE_ACK` carry the 16-byte key. `PROXY_STORE` asks a HighID buddy to fan the same publisher-signed source record out as ordinary `STORE_RECORD` s. The buddy’s ack means “I accepted the proxy request,” not “the DHT stored it.” HighID sources must not ride `PROXY_STORE` — that would bypass the anti-reflection bind (§12.3).

6.8 STORE_BATCH / STORE_BATCH_ACK

Publishing a large library one datagram per (record, target) puts frame count in proportion to records, saturates the link, and trips the receiver’s per-peer rate limit. Records destined for the same peer travel together, so frame count scales with the number of peers.

STORE_BATCH PAYLOAD

```
count u8 ≤ 64, then for each record:
key 16 B
rec_len u16 LE
record N B
rec_sig 64 B
trailing bytes are a framing error — the whole batch is refused
```

STORE_BATCH_ACK PAYLOAD

```
accepted u64 LE bit i set ⇒ record i was stored
```

Acceptance is per record: the storer re-evaluates proximity and capacity. A count alone forced the publisher to treat “some landed” as “all landed” and retire files that were never stored. The datagram budget caps a real batch at roughly twenty minimum-size records; 64 is a decode-side bound matching the width of the ack bitmap.

6.9 FIND_VALUE / FOUND_VALUE**FIND_VALUE PAYLOAD — WIRE VERSION 3 ONWARD**

```
count u8 1..8
keys 16 B × count
start_position u16 LE first record of the key to serve
then, optionally, a constraint block (see below)
```

FOUND_VALUE PAYLOAD — WIRE VERSION 3 ONWARD

```
key 16 B the key this reply is for
next_position u16 LE where a follow-up should resume, or 0 when exhausted
total_available u16 LE matching records the responder holds under the key
record_count u16 LE ≤ 300
for each: len u16 LE + blob (record body || 64-byte publisher signature)
```

The searcher walks the primary (longest) keyword key. Additional keys ride along so a peer that holds records can intersect on `file_hash` locally. On a miss the responder sends `FOUND_NODE` with closer contacts, as in classic Kademlia. A declared count the buffer cannot satisfy is a framing error — the whole frame is refused, so a peer cannot smuggle a truncated payload.

Paging is the searcher’s, not the responder’s. Version 2 had the responder advance a rotation cursor per key, which could not tell “this searcher wants the next page” from “a different searcher wants the first”, so two searchers on one hot key advanced each other’s window and neither saw a contiguous run. Serving is now deterministic: the same request gets the same answer every time.

`next_position` is reported rather than inferred. The packer skips a record too large for the budget *left* on a page and keeps scanning for one that fits, so a page is not always a contiguous run and `start + len` would step over what was skipped — the oversized record would then never be first, never face an empty budget, and never be served, while the responder went on reporting it as held. A page therefore resumes at the earliest record it passed over, which re-sends the ones after it; content-based dedup at the searcher absorbs that. Positions are advisory, since the responder’s list shifts as records expire, so paging may repeat or skip an entry.

Query constraints. **ADDITIVE** A `FIND_VALUE` may append a tag/len/value block carrying minimum size, maximum size, file type, file extension, and any keyword hashes past the eight the count-prefixed run can name (up to `MAX_FIND_VALUE_KEYS_TOTAL` = 23 across both runs). The responder applies it *before* packing, so `total_available` counts matches and a searcher pages through matches rather than through positions it would discard. A key where nothing matches is answered with contacts, exactly as a key we do not hold is.

Additive because `MSG_FIND_VALUE` has always required only a *minimum* length and read its fields at fixed offsets, so bytes past `start_position` have always been valid and ignored. An unconstrained query encodes byte-for-byte as it did before. The block is advisory: a searcher re-applies the same filters at emit, because a peer predating it will answer without having applied them.

Availability is deliberately *not* a constraint. KAD can filter on it because its keyword entries carry a publisher-claimed source count; an Ember record carries none, and the number a search page shows counts distinct publishers across the network, which no single responder can see. A constraint no responder could evaluate honestly is worse than none.

7. Records: keywords and sources

Two record types share a header. The DHT key is not a free field: it is bound to content (`keyword_hash` or `BLAKE3(file_hash)[..16]`) and a STORE under the wrong key is rejected.

```
header = type[1] || keyword_or_source_key[16] || file_hash[16] || ember_file_hash[32]
        || file_size[u64 LE] || publisher_key[32] || timestamp[i64 LE] || name_len[u16 LE]
```

Then: UTF-8 file name (`name_len` bytes), then for source records only the contact block.

TYPE	VALUE	DHT KEY	TTL
RECORD_TYPE_KEYWORD	0x01	BLAKE3(lowercase keyword)[..16]	24 h
RECORD_TYPE_SOURCE	0x02	BLAKE3(eD2K file_hash)[..16]	6 h
RECORD_TYPE_CHANNEL	0x03	Sharded room index / presence / moderation / handoff key	Varies by kind

Keyword media. **ADDITIVE** A keyword record may carry an optional block after its name: `version[1]` then `tag/len/value` triples for duration, bitrate, codec, artist, album and title. It is parsed through to the search row, so an Ember-only hit can fill the Length, Bitrate, Codec, Artist, Album and Title columns that only a server result used to populate.

Additive because `parse_unverified` reads the name from its length prefix and does not length-check a keyword record: an older build parses a longer body correctly and ignores the tail, and a storer relays the bytes it was handed. The name budget charges the block, and the publisher signature covers it, so a relay cannot rewrite it. Publisher-supplied strings are capped and required to be real UTF-8 — they decide sort keys and column widths, and must not become a second name field that disagrees with the file name.

The reader shipped one release before the writer, on purpose: by the time anything published a block, the builds that would receive it already understood it. That is the right order for any additive wire change.

7.1 Keyword hash and tokenization

```
keyword_hash(s) = BLAKE3(s.to_lowercase().as_bytes())[0..16]
```

Publish, and the Search UI, tokenize with eMule's `extract_keywords` so Ember and KAD hash the same words: split on the eMule separators `()[]{}<>,._-!?:;\/"` plus whitespace, keep tokens of **UTF-8 length ≥ 3**, de-duplicate case-insensitively in first-seen order, and drop a trailing 3-byte/3-character token when it is a file extension. Stemming is not performed.

Each surviving token is hashed as above. The Search UI then packs those hashes onto `FIND_VALUE` via `compute_keyword_hashes`: join the tokens with spaces, split on whitespace, drop anything shorter than 2 characters (already gone after `extract_keywords`), sort by length descending (most selective first), and de-duplicate. The first hash is the primary walk key; up to seven more ride along for peer-side `file_hash` intersection.

7.2 Source contact block

Appended after the file name on source records and covered by the publisher signature:

FIELD	SIZE	NOTES
ip	4	IPv4, the address downloaders should dial
tcp_port	2 LE	eD2K client-to-client
udp_port	2 LE	Ember / KAD UDP
flags	1	bit 0 firewalled, bit 1 obfuscation, bit 2 relay-capable
noise_pub	32	Stashed for future native (Noise) dialing
user_hash	16	Optional trailer: publisher's eD2K user hash, present with the buddy block
buddy_ip	4	Optional trailer: HighID buddy the searcher should <code>CALLBACK_REQ</code>
buddy_udp	2 LE	Optional trailer
buddy_noise_pub	32	Optional trailer: buddy's Noise static key
callback_token	16	Optional trailer: publisher-derived token copied into <code>CALLBACK_REQ</code> / <code>CALLBACK</code>

A downloader uses `ip + tcp_port` over the existing eD2K path when the source is not firewalled. Firewalled records append a 70-byte callback trailer (user hash + buddy address + buddy Noise key + token) after the 41-byte contact. HighID records stay 41 bytes. A parser that ignores extra bytes still accepts the contact block; this build reads the trailer when it is present. That is an additive record-body change, not a DHT-frame version bump. **ADDITIVE**

Naming a buddy *requires* that buddy's signed endorsement (`BUDDY_ENDORSE` , 0x14). It was once also possible to name a buddy by quoting its Noise static key, which is on every signed frame that buddy sends — and that was never consent: anyone who had ever heard from a node could name it and spend a replica, a twenty-node fan-out and a callback slot on its behalf. That compatibility path is gone from both sides. A firewalled publisher with no endorsement yet publishes nothing for that file and retries on the next tick, rather than placing a record no searcher would dial.

7.3 Ember file hash

`ember_file_hash` is a 32-byte **streaming BLAKE3 of the file contents** — the same digest `blake3::hash` would produce for the whole file, computed in the same pass as the eD2K and AICH hashes (`Blake3FileHasher`). Search hits, DHT keyword and source records, and `known.met` / library entries can carry it. A download verifies against it whenever one is available: a match shows a green Ember badge (`ember_verified`); a mismatch is a permanent download failure with a red Ember badge and does not reopen parts that already matched the ed2k hash. Deep links without a digest still complete and are hashed for future sharing.

That digest is *not* the dormant native-transfer hash tree. The unused 256 KiB chunk tree in `ember/transfer.rs` hashes each chunk, then BLAKE3 of the concatenated chunk hashes; that root is a different value and is not what DHT records carry.

7.4 Why two TTLs

A source record names an address to download from, so it stops being true the moment that peer goes offline. A keyword record only says a file exists under a word, which stays true whoever is online. Sharing one 24-hour TTL meant a departed peer kept being handed to downloaders for the rest of the day. Publishers re-announce sources every 2 hours, so 6 hours survives two missed republishes while clearing a departed peer four times sooner than a 24-hour source TTL would.

Clock skew tolerance on the signed timestamp is **1 hour** into the future; older than TTL is expired. A replayed older copy cannot replace a newer one: `created_at` is kept and compared.

7.5 Store capacity

CAP	VALUE	BEHAVIOUR WHEN HIT
MAX_RECORDS_PER_KEY	1000	Refuse the newcomer (do not evict incumbents)
MAX_RECORDS_PER_PUBLISHER_PER_KEY	150	Refuse; ~15% of the key, and now KAD's absolute numbers rather than only its ratio
MAX_KEYS	50,000	Evict the furthest key by XOR-distance when the incoming key is closer; refuse a newcomer that is no closer than the furthest held key
MAX_STORE_BYTES	48 MiB	Evict least valuable (furthest from our ID, then nearest expiry)
Sources per IP (adaptive)	8 / 5 / 3	See NetworkScale; firewalled sources attributed to the forwarder

IDENTITY IS FREE

A per-publisher rule can only ever *withhold* capacity, never move it. An earlier attempt displaced whichever publisher held the most slots when a key was full. Publisher identity is a free keypair, so an arrival with no slots always outranked an established publisher. A few hundred keypairs stripped a healthy keyword to one record per honest publisher, after which the key admitted no one ever again. Filling an empty key with seven identities is still possible; bounding that needs something scarcer than a keypair.

The publisher cap is network-wide in effect: every storer applies the same cap to the same publisher key, and the same allowance applies to their own local store. It was 45 of 300 through wire version 2 — so a user sharing 200 files with a word in common got 45 of them findable under that word *anywhere*, 30% of what KAD serves. Raising both to KAD's own 150 of 1000 waited on searcher-driven paging (§6.9): while a peer could only ever serve its first window, the extra stored records had no way to reach a searcher and the one certain effect would have been more replication traffic. `MAX_STORE_BYTES` is unchanged, so per-key capacity tripled without raising what the process may hold resident.

`MAX_KEYS` is not a hard admission freeze. When the map is full, expired keys are dropped first; otherwise the furthest key from this node (XOR-distance) is given up, and only when the incoming key is closer than it. A flood of distant keys therefore cannot deny keys this node is responsible for. Without a local ID there is no notion of responsibility, and a full map refuses newcomers.

7.6 What a FOUND_VALUE blob is

On the wire, each record in `FOUND_VALUE` is `record_body || signature[64]`. The searcher verifies the publisher signature, checks that `keyword_hash` matches the queried key, and drops junk. The per-node result budget is charged on blobs *offered*, not only those accepted, so a peer that sends nothing but junk still hits its own limit.

8. Iterative lookup and search

8.1 Shortlist walk

Both `FIND_NODE` and `FIND_VALUE` use an iterative shortlist, $\alpha = 5$ outstanding queries, up to $k = 20$ closest contacts as the frontier.

- 1 Seed the shortlist from the routing table (verified contacts preferred). Local store may pre-fill `FIND_VALUE` results, capped at half the result budget so a well-stocked node cannot finish the search from itself alone.
- 2 Query up to α pending contacts. A node may be asked twice in one search (one loss is not proof it is gone); a third miss marks it failed.
- 3 On `FOUND_NODE`, merge closer contacts. On `FOUND_VALUE`, verify blobs and merge records. Those two outcomes are independent: treating “no closer node” as “query not accepted” used to stall every value lookup until timeout.
- 4 `FIND_NODE` converges after α consecutive responses that bring nothing closer, but only once at least $k/2$ nodes have answered. `FIND_VALUE` does *not* use this rule: every extra node is another chance at a record the closer ones did not hold. The walk continues until the shortlist is exhausted, the 300-result budget is full, or 60 seconds elapse.

LIMIT	VALUE
<code>MAX_ACTIVE_SEARCHES</code>	64
<code>SEARCH_TIMEOUT_SECS</code>	60
<code>MAX_SEARCH_RESULTS</code>	300
<code>MAX_LOCAL_SEED_RESULTS</code>	150 (half the budget)
<code>MAX_RESULTS_PER_NODE</code>	75 (quarter of the budget)
<code>MAX_QUERY_ATTEMPTS</code>	2
Queued-query timeout (cold handshake)	12 s

The 12-second queued-query timeout covers a Noise_XX 2-RTT setup plus the query round trip. Charging a first-contact hop the ordinary budget failed the contact before it could answer — and on a cold table every contact is a first contact.

8.2 Multi-keyword search

Intersection is **sparse**: missing secondary keys are skipped, plus a filename match at emit time. It is not a strict worldwide AND of every keyword. Product search already tokenizes with `extract_keywords`

(§7.1), so 1- and 2-byte tokens never become DHT keys. Successive queries rotate which window a storer serves (§8.4).

The Search page exposes an **Ember Only** method. **Global** queries Ember alongside KAD and servers, merging into one de-duplicated list. With KAD and eD2K both offline, Global falls back to Ember alone. Downloads also resolve sources on EmberDHT in addition to KAD and servers.

8.3 Streaming results

Ember used to buffer everything and emit on completion, so on a cold table a user waited most of the 60-second cap while KAD hits were already on screen. As of 1.5.3 the keyword search carries a cursor into an append-only result list. A 1-second sweep emits everything past it on the same cadence as KAD: the first record immediately, then every 20. Locally seeded records therefore reach the UI on the first tick.

Batches run through hash de-duplication against what KAD already streamed, so a hash KAD found arrives as an availability update rather than a duplicate row. Only the batch flagged final clears `ember_pending`, and the completion batch is still queued when empty so that happens exactly once. The timeout backstop that reaps an expired search runs *before* the streaming and emit steps in the same sweep, which is what stops a reaped search from being streamed after its `search-complete`.

8.4 Serving window and paging

A peer answers a keyword query with whatever fits the 1235-byte record budget: about five records for a bare keyword record, four for one carrying media, two in the worst case. Through wire version 2 packing filled from the front of insertion order, so the oldest five were the only ones a node would ever serve; a rotation cursor was then added, but the responder owned it and could not tell one searcher from another (§6.9).

Since v3 the searcher owns the offset, so a walk pages a well-stocked node until the key is exhausted. Paging is the one mechanism here where a *responder* influences how many queries we send, so the searcher bounds it independently of what `total_available` claims: each follow-up must name an offset strictly past the one it answered, and both the per-node result allowance and the page ceiling are two-tier.

While the shortlist still holds an unqueried hop — or any query is outstanding — one peer may offer `MAX_RESULTS_PER_NODE` (75, a quarter of the 300-file budget) over up to `MAX_PAGES_PER_NODE` (25) follow-ups. Once neither is true there is no hop left for extra records to crowd out, so a lone storer may spend what remains of the whole budget, over up to `MAX_PAGES_PER_NODE_EXHAUSTED` (100) pages. That second page tier has to be *earned*: past the base ceiling a node keeps paging only while it sustains `MIN_RECORDS_PER_PAGE_TO_CONTINUE` (2) records per page on average, so a peer answering one record at a time while claiming a huge total stops at the base ceiling.

Blob de-duplication on the searcher is content-based, so a re-sent record costs bandwidth but not a slot of the per-node allowance, which is charged per distinct blob. Local seeding does no packing at all.

Truncation is still counted: `ember_dht_found_value_truncated` and `ember_dht_found_value_withheld`, which is how to read how far the datagram ceiling binds on real keys. Read *withheld* as records past this page's window that it has not served. It once counted from the rewind resume point, which included records the same page had just put on the wire; a page that rewinds but still reaches the end of its key now reports zero withheld and does not increment `truncated`, because it truncated nothing.

9. Publish and replication

9.1 Publisher cycle

Shared files are published as keyword records (one per `extract_keywords` token) and one source record, and republished on a cycle to stay alive. Publishing starts once the node has contacts. The Library marks a file with an **Ember** badge once a source record is placed — that is the point at which other Ember users can actually fetch it.

CYCLE	INTERVAL	NOTES
Source republish	2 h	Persisted in known.met as FT_EMBER_SOURCE_PUBLISH (0xE4)
Keyword republish	12 h	Persisted in known.met as FT_EMBER_KEYWORD_PUBLISH (0xE5)
Rendezvous advertise	5 h	Only while reachable and already publishing something
Publish timeout	30 s	Wait for STORE_ACK
MIN_STORE_NODES	5	Minimum replicas the publisher aims for
MAX_ACTIVE_PUBLISHES	128	In-flight publish operations

Keyword publish rate per maintenance tick scales with library size, clamped between 2 and 96 **files** per tick (not records), aiming to finish a full pass inside the 12-hour cycle at an estimate of 8 keywords per file. Source publish is similarly paced (5–256 files per tick) so a large library is not slammed in one tick after a restart.

The last successful source-publish and keyword-publish are written to `known.met` (`0xE4` / `0xE5`). On start, a stamp still inside its interval (2 hours for sources, 12 hours for keywords) is hydrated back to an `Instant` ; a stamp older than the interval, or one that cannot be represented because the process has not been up that long, is omitted and the file is due immediately — republish-too-eager, the safe direction. Previously both schedules were keyed on `Instant` alone, so every restart marked the whole library as never-published.

9.2 Storer-side replication

Nodes that hold a record re-send the identical signed bytes to the current closest nodes every **2 hours** (`EMBER_RECORD_REPUBLISH_SECS = 7200`), up to 200 records per 60-second maintenance tick, using `STORE_BATCH` where possible.

Replication **cannot extend a record's lifetime**. Expiry is derived from the publisher's signed creation timestamp; every recipient computes the same absolute death time. What it buys is churn coverage: copies reaching nodes that joined since the publisher's last round. Each record already has up to 20 replicas and lives at most 24 hours (6 for sources). Hourly replication was Ember's single largest traffic item — roughly 48,000 frames an hour at 200 records × 20 replicas — about double the entire publish load. Two-hourly halves that.

Firewalled source records are not republished by storers: their declared address is unverified, and repeating it would amplify a lie. A `republish_due` flag, rather than an old `Instant` , marks records whose

previous attempt never made it onto the wire — `Instant::now() - 24h` is not representable on a machine that has been up for less than a day, and the saturating fallback stamped the record as *just* republished.

Each maintenance cycle logs an `Ember publish cycle` heartbeat and an `Ember replication cycle` heartbeat (due, selected, queued, re-armed, leftover backlog).

10. Bootstrap and joining

There is no central bootstrap and no shipped address list. A cold node gets in through the following, in rough order of who arrives first.

- 1 **The KAD rendezvous key.** Ember nodes advertise themselves under one fixed KAD key as an ordinary source record carrying their Noise pubkey tag. A node with a near-empty table runs a plain source lookup there. Re-advertised every 5 hours (matching KAD's source TTL) and only while the node is reachable and already publishing something — a leecher never generates publish traffic purely to list itself. Lookups are spaced 10 minutes apart and stop once the table reaches one k-bucket of verified contacts. Seed string: `ember-dht-rendezvous-v1`, hashed with MD4 into a KAD ID. The derivation is versioned so the key space can be sharded later without a flag day. Health is surfaced on the Ember page as `ember_dht_rendezvous_*` (listed / lookups / empty).
- 2 **The KAD bridge.** Ember peers noticed in ordinary KAD traffic are DHT-pinged so their signed PONG folds them into the routing table. Capped per maintenance cycle; quiet above 20 verified contacts.
- 3 **eD2K client-to-client sessions.** Peers that advertise the Ember capability bit over a normal eD2K transfer are cached with their UDP port and bridged via Noise_XX when no static key is known. This is the path for a client running with no KAD at all.
- 4 **DHT gossip and `nodes_ember.dat`.** FOUND_NODE / PEER_LIST / ANNOUNCE_PEER, plus up to 200 persisted contacts, once the node has been online before. Persisted contacts load as unverified leads.

COLD JOIN WITHOUT EMULE

Every path except `nodes_ember.dat` presupposes either a live KAD connection or an eD2K transfer with an Ember-capable peer. A first-run user with KAD off and no servers has no way in. Ember rides eMule's bootstrap by design. Seed lists are deliberately not planned. For a local two-node test where neither side can reach KAD, the harness command `add_ember_dht_contact` is the only way to introduce two nodes directly.

10.1 Explicitly not planned

- Hardcoded `seeds.txt` or DNS SRV seed lists.
- A rendezvous-hosted bootstrap pool — it leaks an identity-to-IP roster of every participant to whoever runs the server. The pool, its endpoint, and the pinned key it verified were deleted because the server and client never agreed on the envelope format.

The Friends rendezvous server is still used for friend NAT traversal and relay. That is a different protocol and a different trust boundary.

11. NAT, observed address, and firewalled peers

11.1 Observed-IP voting

A `PONG` may echo the address the ping was received from. Votes are counted per reporter `/24`, expire after 15 minutes, and require **3 distinct public nets** before the address is confirmed. Private/loopback reporters and private reported addresses are ignored so LAN Sybils cannot vote. At most 64 distinct reported addresses are tracked; the least-recently-updated is dropped.

Without expiry the three-vote threshold was cumulative over the whole process lifetime, so a stale address stayed qualified indefinitely and a genuine address change could not displace it.

11.2 Firewalled publishing and consume

A LowID / firewalled node sets `SOURCE_FLAG_FIREWALLED` on its source record, names one HighID `PROXY_STORE` target in the callback trailer, and asks *that* contact (only) to fan the record out. Overlay `STORE_BATCH` of that firewalled record waits for the buddy's matching `PROXY_STORE_ACK`, so `FIND_VALUE` cannot advertise a buddy that has not yet accepted the record. Storers attribute the record to the forwarder for the per-IP source cap, because the declared address is unverified and an attacker could otherwise invent a new address per record — each getting a fresh quota.

Consume is the KAD callback analogue. A reachable searcher that finds a firewalled record with a buddy trailer sends `CALLBACK_REQ` to that buddy over the Ember Noise session, carrying the token from the signed trailer. The buddy forwards `CALLBACK` only if it recently accepted `PROXY_STORE` from the named publisher, copying the token through. The publisher accepts `CALLBACK` only from a buddy that answered `PROXY_STORE_ACK` with the request id it was sent *for that file*, and only when the token matches the one it published, then connects eD2K TCP back to the searcher's *observed* address (the UDP source of `CALLBACK_REQ`, copied by the buddy — never a claimed IP in the request). Firewalled Ember DHT contacts are not registered in SourceManager (pending promotion would otherwise TCP-dial the claimed NAT IP). `WaitCallbackKad` rows are kept out of TCP reask and pause/resume seeding; only `CALLBACK_REQ` retries them. Consume uses the same TCP-firewalled predicate as publish (LowID or KAD/server `Firewalled`), not the UPnP-pessimistic startup flag. A named buddy that is banned, filtered, or special-use parks the source — including Searching-only pending downloads — rather than dropping it. When the searcher is also TCP-firewalled, ingest does not send `CALLBACK_REQ`; firewalled Ember records set `SOURCE_FLAG_RELAY_CAPABLE`, and that path starts the same Ember punch/relay broker KAD uses for Ember-capable LowID sources rather than leaving both sides parked. The broker still needs admitted ERAT candidates.

LowID publishing through buddy `PROXY_STORE` is implemented but not yet exercised end to end on a live network. Consume is implemented on the same path: the buddy table is populated when a `PROXY_STORE_ACK` matches a request we sent, so a network that is already forwarding publishes can bounce callbacks without a second handshake.

12. Abuse resistance

12.1 NetworkScale

Every anti-abuse limit has the same tension: strict values keep one host from occupying the table or the store, but on a small network a legitimate peer looks a lot like an attacker. Limits are derived from **verified** contacts — gossip is cheap and proves nothing. Counting leads would let anyone who can talk at us drive the limits to their strict tier while we had reached almost nobody.

TIER	VERIFIED CONTACTS	PER IP	/24 GLOBAL	/24 PER BUCKET	SOURCES/IP	STORES/MIN
Bootstrap	< 10	8	20	5	8	300
Small	10...79	4	16	4	5	200
Established	≥ 80	2	10	3	3	120

Established per-IP is 2 rather than KAD's 1 because two genuine instances behind one NAT is ordinary, and unlike KAD, Ember can tell them apart cryptographically. Established per-subnet matches KAD (10 global, and the per-bucket cap converges on 3). Store rate is counted in **records**, not frames: every record costs two signature verifications whether it arrives alone or batched. Charging per frame let one dense batch buy many times the admitted work of a well-behaved publisher.

Tightening never evicts contacts admitted under a looser tier, so whatever a cold start allows, an adversary present at that moment keeps for the life of the process — in the bucket that matters most, since geometric occupancy puts half of everyone there. The Bootstrap per-bucket cap is therefore kept $\leq k/4$.

12.2 Frame rate limits

Applied after the Noise session is up. A two-bucket sliding window approximates a trailing window so a sender cannot straddle a reset and spend twice the limit. Effective sustained rate is about twice the named constant because the window is enforced over a trailing half.

BUDGET	WINDOW	NAMED CAP	KEYED ON
All DHT frames	1 s	40	IP
FIND_NODE / FIND_VALUE / CALLBACK_REQ	10 s	60	IP (shared behind NAT)
STORE records	60 s	NetworkScale	Verified node ID when known

Lookup budgets stay on the address because resolving identity for every datagram would put a routing-table scan in front of the gate that exists to make junk cheap to reject. STORE budgets use the verified node ID so peers sharing a NAT do not throttle each other.

12.3 Anti-reflection

HighID / direct source records must claim the observed Noise sender IP. Otherwise a peer could point downloaders at a third-party victim (and Ember would be a traffic reflector). Firewalled sources are exempt: they may be stored by a HighID buddy, or from a NAT mapping that differs from the STUN hint. Authorship is still bound by the publisher Ed25519 signature.

Return-routability is required before a node spends anything substantial on a request (the XX cookie is the handshake-level form of the same idea).

12.4 STORE signature replay

Identical STORE frames (same publisher signature) collapse for 60 seconds so a retransmit storm cannot re-verify the same blob forever. A replay only counts if the store still holds that signature: the cache is never cleared by eviction, and treating a displaced record as a replay made `STORE_BATCH` set the accepted bit for a replica that was gone, after which the publisher retired the file.

Cache size 50,000; swept at most once per TTL. An unconditional size check meant a single 64-record batch could walk a 25,000-entry map 64 times.

12.5 Other gates

- Private IPs and user range filter on routing-table admission.
- No reputation scoring on gossip itself — admission is bounded by diversity caps, and a lead that never answers a liveness ping faults out after 3 failed queries.
- Store refusals are counted by cause: verify, signature, timestamp, anti-reflection IP, per-IP cap, publisher cap, per-key cap, proximity.

13. Maintenance

A 60-second tick drives bucket refresh, liveness pings, gossip, publisher cycles, and storer replication. Each task is internally gated on a longer interval, so the cadence itself is cheap.

TASK	INTERVAL / BUDGET	NOTES
Tick	60 s	EMBER_MAINT_INTERVAL
Disconnect / re-bootstrap	inbound silence > 20 min	Re-arms the KAD bridge, rendezvous lookup, and self-lookup rather than sitting on a dead table
Stale-contact purge	verified unheard > 2 h	Contacts still needed by an in-flight search are spared. Unverified leads are not purged this way (last_seen is 0)
Self-lookup	180 s cold / 20 s if ≥ 10 verified; repeats every 4 h	FIND_NODE for our own ID, to fill close-to-home buckets
Bucket refresh	idle > 3600 s, max 3 FIND_NODE / tick	Random target in the idle bucket
Liveness ping	unheard > 600 s	8/tick steady; 32/tick while verified < 20. Scales to cover the table inside the stale window, capped at 32. 8 s ping timeout.
Lead ping reserve	1 in 4 of the budget	When verified contacts could spend the whole budget, keep promoting gossip
ANNOUNCE_PEER	2/tick; 16/tick while starved	Highest-yield cold frame; self-limits above 20 verified
Storer republish	2 h, 200 records/tick	STORE_BATCH to current closest
KAD bridge	while verified < 20	Same ping budget as starved liveness
Rendezvous lookup	every 10 min while verified < 20	Stops at one k-bucket

A ping is the only way an unverified lead becomes usable, so the starved budget is also the join rate. Eight a minute is a sensible steady-state trickle and a poor join. Verification is cheap — one small frame each.

14. Persistence

14.1 nodes_ember.dat

NODES_EMBER.DAT · MAGIC EMB3 (0X454D4233 LE) · VERSION 1

```
magic(4) || version(1) || count(u16 LE) ||
for each: node_id(16) || addr_type(1) || ip(4 or 16) || port(2 BE) || noise_pub(32) || ed25519_pub(32) ||
last_seen(i64 LE)
```

- Up to 200 contacts. Written atomically (tmp + write + sync + rename + dir-sync).
- An empty table never overwrites an existing file — a momentary empty table would otherwise leave the next launch with no way back in.
- On load, `last_seen` is discarded and set to 0. Restoring it would make every entry look verified the moment it loads, so the staleness purge would delete the whole bootstrap set before a single ping went out. Zero also sorts them first for liveness probing.
- Node ID is re-derived from the persisted Ed25519 key; corrupt keys are dropped.

14.2 store_ember.dat

STORE_EMBER.DAT · MAGIC EMS1 (0X454D5331 LE) · VERSION 1

```
magic(4) || version(1) || count(u32 LE) ||
for each: key(16) || created_at(i64 LE) || publisher_key(32) || signature(64) || ip_present(1) || ip(4,
optional) || data_len(u32 LE) || data
```

- Up to 20,000 records. Written on shutdown only — a periodic write of several megabytes was a disk hitch every few minutes. Abnormal exit falls back to replication refilling the store within two hours (the storer republish cadence).
- Only `data` and `signature` are load-bearing on the way back in. Key, publisher, and expiry are re-derived from the signed body. Believing the unsigned fields would let anything that can write the file place a genuine record under the wrong key or with a life it was never granted.
- Records already past TTL are refused, so the file cleans itself up. Bodies larger than 4096 bytes are corruption.

14.3 Other durable state

LOCATION	WHAT
known.met tag 0xE4	Last successful Ember source-publish per file
known.met tag 0xE5	Last successful Ember keyword-publish per file
ember_dht_highwater.json	Daily and all-time high-water of verified contacts

15. Comparison with eMule KAD

EmberDHT was compared against this repository's own KAD stack, constant for constant. On design it already matches or exceeds KAD operationally in the rows below. Remaining gaps are in §16.

EMBERDHT

- $k = 20$ replicas
- $\alpha = 5$ concurrent FindNode
- Source TTL 6 h vs 2 h republish (3× margin)
- Keyword TTL 24 h
- Replacement cache
- Storer-side replication
- Store survives restart
- Ed25519-signed frames and records
- Noise transport
- Adaptive abuse limits
- Rich diagnostics (truncation, reject causes, search quality, high-water)
- Buddy endorsement + `CALLBACK_REQ` for firewalled consume
- Searcher-owned `FIND_VALUE` paging
- Media on keyword records
- Peers advertise the wire versions they speak

KAD (THIS REPO)

- $k = 10$ replicas
- $\alpha = 1$ for FindNode
- Source TTL 5 h vs 5 h republish (no margin)
- No replacement cache
- No storer-side replication
- No signed overlay identity
- Cleartext UDP (obfuscation optional)
- Fixed caps (1 contact/IP, 10/subnet, 3 sources/IP)
- 150 of 1000 records per sender per key — now matched
- Buddy hash + callback for firewalled consume
- Keyword search carries query constraints — now matched

Roughly half the round trips per lookup come from $\alpha = 5$ against KAD's 1 for `FindNode`. Replica count is double. The diagnostic surface is already richer. Firewalled consume (§11.2) now has an Ember callback path matching KAD's buddy relay, and Ember DHT ingest starts the same LowID↔LowID broker KAD uses for Ember-capable sources. The largest product gap is that bytes still move over eD2K.

16. Current limits and planned work

16.1 Known limits (user-facing)

- **Content still moves over eD2K.** Native 256 KiB chunks and a BLAKE3 hash tree exist but nothing imports them. Downloads still verify the streaming BLAKE3 pin when one is present: mismatch is a permanent failure with a red Ember badge.
- **Bootstrap depends on eMule** except for a previously persisted contact file.
- **A version bump still partitions** peers running a build older than the advertised range (§6.1). The refusal is clean and counted, and the Ember page raises a banner, but the peer that needs to update cannot decode anything we send.
- **Multi-keyword search is approximate** — sparse intersection plus filename match, not a worldwide AND. A query may name up to 23 keywords, but only the first eight travel in the run every build reads; the rest ride the constraint block, so a peer predating it intersects on eight and leaves the remainder to the filename match.
- **A few records per reply** — five for a bare keyword record, four with media, two in the worst case — but a searcher can now page a node until its key is exhausted (§8.4).
- **150 records per publisher per keyword, network-wide.**
- **Media columns fill only after a republish.** A record published by a build predating the media block simply has none, so those columns stay empty until that publisher re-announces.
- **Not yet exercised end to end:** LowID PROXY_STORE / callback, cold join from an empty contact file with no KAD, republish across a full record TTL on a large library.

16.2 Shipped since wire version 2

Both items this section previously listed as waiting on a bump have landed, as wire version 3, together — one bump paid for both, and the capacity raise was only worth having once paging could serve the extra records.

1. `start_position` on **FIND_VALUE / FOUND_VALUE** — searcher-owned paging, replacing the responder-side rotation cursor (§6.9, §8.4).
2. `node_id` **dropped from wire contacts** — 87 → 71 bytes, 14 → 17 contacts per FOUND_NODE (§6.4).
3. `MAX_RECORDS_PER_PUBLISHER_PER_KEY` **raised to KAD's 150 of 1000**, unblocked by the above (§7.5).

Everything else added since needed no bump, because each went where an existing decoder does not look — after the fields a payload's parser reads at fixed offsets, or after a record's length-prefixed name: query constraints and keyword media, the callback and endorsement message pair, channel records, and the advertised version range. A change that must alter an *existing* field still has no path but a bump.

16.3 Future, non-blocking

- Shard the rendezvous key space. Its trigger is now observable: the storer load reported for the rendezvous key, since a lookup's returned-peer count saturates at the source-search stop threshold (20) and can never see a 1000-entry key.
- Richer keyword indexing (stemming) if recall lags KAD on real libraries. Now measured per search as files found by both DHTs, by KAD only, or by Ember only.

- Route a future wire change around peers that have not advertised a range, rather than shipping it on a flag day (§6.1).
- Wire the native transfer path (`ember/transfer.rs`).
- Longer fuzz / property tests and overnight soak jobs.

Gossip is now scored rather than merely rate-limited: an introducer's probes are rationed by how many of the leads it gave us went on to answer, though it is never consulted while the table is starved and never refuses a contact outright. Sybil pressure on the *store* remains bounded by address diversity rather than by identity, since every per-publisher share is keyed on a free keypair; the per-address STORE ceiling is where a rotating-identity flood becomes visible, and it is counted.

16.4 Dormant native transfer

`ember/transfer.rs` specifies a stream of `REQUEST_CHUNKS`, `CHUNK_DATA`, `FILE_INFO`, and `TRANSFER_COMPLETE` over 256 KiB chunks with a BLAKE3 hash tree whose root is a separate digest from the streaming BLAKE3 already carried on DHT records. Nothing in the tree imports this module. Wiring it is the largest remaining piece if the goal is a network that does not need the eMule wire at all.

Appendix A. Constant tables

Values as compiled in Ember 1.6.3. Source files in parentheses.

A.1 Overlay

CONSTANT	VALUE	FILE
EMBER_DHT_VERSION / MIN	4 / 4	dht/mod.rs
K_BUCKET_SIZE	20	dht/mod.rs
ALPHA	5	dht/mod.rs
ID_BITS	128	dht/mod.rs
MAX_CONTACTS_PER_RESPONSE	20 (17 IPv4 fit)	dht/mod.rs
CONTACT_TIMEOUT_SECS	600	dht/mod.rs
MAX_FAILED_QUERIES	3	dht/mod.rs
EMBER_MAGIC	0xEB 0x3E	transport.rs
CONTROL_VERSION	0xC1	transport.rs

A.2 Datagrams and payloads

CONSTANT	VALUE	FILE
MAX_EMBER_DATAGRAM_BYTES	4096	transport.rs
MAX_UNFRAGMENTED_DATAGRAM	1400	dht/messages.rs
FRAME_OVERHEAD	120	dht/messages.rs
TRANSPORT_OVERHEAD	27	dht/messages.rs
MAX_UNFRAGMENTED_PAYLOAD	1253	dht/messages.rs
MAX_FOUND_VALUE_RECORD_BYTES	1235	dht/messages.rs
MAX_STORE_RECORD_BYTES	1169	dht/messages.rs — FOUND_VALUE singleton pack budget (1235 – 2 – 64)
MAX_FOUND_VALUE_BLOB_BYTES	1233	dht/messages.rs — body + 64-byte signature
MAX_FIND_VALUE_KEYS	8	dht/messages.rs
MAX_FIND_VALUE_EXTRA_KEYS	15 (constraint block)	dht/messages.rs
MAX_FIND_VALUE_KEYS_TOTAL	23	dht/messages.rs
CONTACT_WIRE_LEN_IPV4	71	dht/messages.rs
MAX_STORE_BATCH_RECORDS	64	dht/messages.rs
MAX_FOUND_VALUE_RECORDS	300	dht/messages.rs

A.3 Time and maintenance

CONSTANT	VALUE	FILE
EMBER_MAINT_INTERVAL	60 s	network/mod.rs
EMBER_BUCKET_REFRESH_SECS	3600	network/mod.rs
EMBER_RECORD_REPUBLISH_SECS	7200	network/mod.rs
EMBER_SOURCE_REPUBLISH	2 h	network/mod.rs
EMBER_KEYWORD_REPUBLISH	12 h	network/mod.rs
EMBER_KEYWORD_PUBLISH_MIN/MAX_PER_TICK	2 / 96 files	network/mod.rs
EMBER_SOURCE_PUBLISH_MIN/MAX_PER_TICK	5 / 256 files	network/mod.rs
KEYWORD_RECORD_TTL	24 h	dht/store.rs
SOURCE_RECORD_TTL	6 h	dht/store.rs
CLOCK_SKEW_TOLERANCE_SECS	1 h	dht/store.rs
EMBER_RENDEZVOUS_REPUBLISH_SECS	5 h	network/mod.rs
EMBER_RENDEZVOUS_LOOKUP_INTERVAL_SECS	10 min	network/mod.rs
EMBER_SEARCH_QUEUED_QUERY_TIMEOUT	12 s	network/mod.rs
EMBER_SELF_LOOKUP_FIRST_DELAY_SECS	180 s (20 s if ≥ 10 verified)	network/mod.rs
EMBER_SELF_LOOKUP_REPEAT_SECS	4 h	network/mod.rs
EMBER_DISCONNECT_SECS	20 min	network/mod.rs
EMBER_CONTACT_STALE_SECS	2 h	network/mod.rs
SESSION_TIMEOUT	900 s	transport.rs
MAX_SESSIONS_PER_ADDR	4	transport.rs
STORE_SIG_REPLAY_TTL	60 s	dht/engine.rs
MIN_OBSERVED_IP_VOTES	3 / 15 min	dht/observed.rs

A.4 Persistence and identity files

CONSTANT	VALUE	FILE
EMBER_PERSIST_MAX_CONTACTS	200	network/mod.rs
EMBER_PERSIST_MAX_RECORDS	20,000	network/mod.rs
NODES_EMBER_MAGIC	EMB3	dht/bootstrap.rs
STORE_EMBER_MAGIC	EMS1	dht/bootstrap.rs
FT_EMBER_SOURCE_PUBLISH	0xE4	known.met
FT_EMBER_KEYWORD_PUBLISH	0xE5	known.met
EMBER_RENDEZVOUS_SEED	ember-dht-rendezvous-v1	kad/publish.rs

Appendix B. Code map

AREA	LOCATION
DHT engine, routing, store, search, publish, wire	<code>src-tauri/src/network/ember/dht/</code>
Protocol constants (k, α , version, IDs)	<code>dht/mod.rs</code>
Frame encode/decode, message types, size budgets	<code>dht/messages.rs</code>
IO-free inbound handling, proximity, PROXY_STORE	<code>dht/engine.rs</code>
128-bucket table, replacement cache, admission	<code>dht/routing.rs</code>
Signed records, capacity, TTL, serve cursor	<code>dht/store.rs</code>
Iterative lookup, FIND_VALUE key packing	<code>dht/search.rs</code>
Keyword/source record builders	<code>dht/publish.rs</code>
Filename / query tokenization	<code>kad/publish.rs</code> · <code>extract_keywords()</code>
Streaming BLAKE3 file digest	<code>ember/crypto.rs</code> · <code>Blake3FileHasher</code>
Adaptive abuse limits	<code>dht/scale.rs</code>
Per-IP / per-peer rate limits	<code>dht/protection.rs</code>
Observed-IP voting	<code>dht/observed.rs</code>
<code>nodes_ember.dat</code> / <code>store_ember.dat</code>	<code>dht/bootstrap.rs</code>
Noise transport, magic, sessions	<code>ember/transport.rs</code>
Node ID, sign/verify, hash binding	<code>ember/crypto.rs</code>
Dormant native transfer	<code>ember/transfer.rs</code>
Publish/search drivers, maintenance, KAD bridge	<code>network/mod.rs</code>

AREA	LOCATION
Rendezvous KAD key	<code>kad/publish.rs</code> · <code>ember_rendezvous_key()</code>
User-facing status	Ember Network page (<code>/ember</code>), status bar
Settings	Settings → Network · <code>ember_native_enabled</code> (always on)
Dev harness	<code>add_ember_dht_contact</code> , <code>ember_dht_ping_peer</code> , <code>ember_dht_find_node</code> , <code>ember_dht_iterative_find_node</code> , <code>ember_dht_publish_keyword</code> , <code>ember_dht_find_value</code> , <code>ember_dht_run_maintenance</code>
Standing work log	<code>docs/ember-dht.md</code>

EmberDHT Protocol Specification · Wire version 4 · Ember 1.6.3 · September 2026. Derived from the canonical implementation in the ember-p2p repository. Licensed under the GNU General Public License, version 3.

This document describes behaviour as implemented, and is not a promise that the wire will remain unchanged. Version 3 carried pagination and contact-list compaction; version 4 bound a frame signature to the session it arrives on. Everything since has been additive — query constraints, keyword media, the callback and endorsement pair, channel records, and the advertised version range — and additive means a peer that does not speak it parses the frame correctly and ignores the addition.

Implementers targeting interoperability should treat `EMBER_DHT_MIN_VERSION..=EMBER_DHT_VERSION` as the only supported range, read that range from the block on a `PING` or `PONG` where a peer offers one, and treat its absence as “this peer speaks version 4 and nothing else” rather than as consent to anything newer.